



Abstract

Early detection of breast cancer is an essential matter. That is why diagnostic imaging is a fundamental part of clinical practice. Medical images are an essential tool on many occasions to make a good diagnosis or conclude a clinical study successfully. However, the number of clinical images and the experts needed for their interpretation make this challenging.

As future engineers, we tried to face this problem from a technical point of view. We used image processing tools and machine learning algorithms applied to a small dataset with the goal of automatizing the detection of breast cancer from mammographies.

Radiomics Image Analysis

2nd Trimester

Image processing tools were used in order to purify the given images in combination with a segmentation algorithm for the purpose of preparing the dataset. The open-source package *pyradiomics* was implemented in order to extract numerical features from the images. Straightaway two machine learning methods (supervised using Weka, and unsupervised using PCA) were performed in order to obtain a good classification of the data.

Finally, the implemented methodology made the study not accurate enough to be considered clinically viable because of the small number of samples.

1.Introduction

In 2020, more than 34,000 cases of breast cancer were detected in Spain. According to the Institute of Investigation for Breast Cancer, 30% of diagnosed cancers have their origin in the breast [1]. Therefore, the statistics say that 1 in 8 women could experience this disease during her life.

The detection of this cancer in its early stages is essential and most of the time a matter of life and death. If the tumor is detected in an advanced stage and the metastasis has begun, the life expectancy of the patient after 5 years is approximately 20%. In most cases, there is no cure for metastatic cancer, just palliative treatment [2].

Cancer occurs when a cell disobeys the cell laws and starts proliferating in an accelerated manner and without control. In the breast case, this proliferation happens in the epithelial glandular cells [3]. From here, the uncontrolled growth could move to the lymph nodes and from there, to the rest of the body. However, early detection helps to avoid this.

There are many techniques used in the clinical area. It goes from mammography and ultrasound to core-needled biopsy. This last one is used in order to determine the malignance nature of the tumor. Nevertheless, in this study, we are going to focus on the detection of cancer from mammographies.

In this work, we are given a dataset from the Mammographic Image Analysis Society (Minimammographic Database) [4]. This dataset contains 24 mammographies and an excel which indicates the type and the diagnosis (*Figure 1*). The letters of type go for:

- F: Fatty
- G: Fatty-glandular
- D: Dense- glandular

This study aims to find patterns between characteristics from mammographies that allow us to diagnose and differentiate between normal and spiculated malignant cases having just the mammographies of the patients.

Subjects	Type	Diagnose
mdb003	D	NORM
mdb004	D	NORM
mdb006	F	NORM
mdb007	G	NORM
mdb008	G	NORM
mdb009	F	NORM
mdb011	F	NORM
mdb014	G	NORM
mdb016	G	NORM
mdb018	G	NORM
mdb020	G	NORM
mdb022	G	NORM
mdb024	G	NORM
mdb026	F	NORM
mdb027	F	NORM
mdb029	G	NORM
mdb148	F	SPIC M
mdb178	G	SPIC M
mdb179	D	SPIC M
mdb181	G	SPIC M
mdb184	F	SPIC M
mdb186	G	SPIC M
mdb202	D	SPIC M
mdb206	F	SPIC M

Figure 1. Original dataset in .xls format.

We are going first to perform a pre-processing of the images since some of them contain labels identifying patients and can be considered noise in the study. Afterward, we are going to segment the regions we considered of interest with *MATLAB* [5].

After this previous work, we will use different software in order to extract all the radiomics of the images with *Python 3.8.7* [6] and *PyRadiomics* library [7], which will be our way to compare the information between the different mammographies. There are many softwares that allow us to perform this work, and we are going to select just the radiomics we find out to be relevant for the diagnosis.

We will then analyze the feature extraction work with Machine Learning (ML) algorithms, such as Decision Trees with *Weka 3.8.6* [8]. Additionally, we will make a Principal Components Analysis (PCA) with this data [9]. This process will tell us if there exists an automatic manner to classify the images into 2 groups (normal and pathological). Finally, we will discuss these results and extract some conclusions.

2. Methodology

Before extracting the features of interest from the given mammographies, a previous pre-processing, and a good segmentation of the structure of interest (SOI) from the images, are needed to ensure a higher performance when in future feature extraction. The following strategy explained in sections 2.1 and 2.2 is all conducted in MATLAB, with the use of *Image Processing Toolbox*. See *Appendix. Processing and Segmentation Code* to look at the code used for the first two sections.

Once we have the pre-processed and segmented images, feature extraction is carried out, explained in section 2.3.

2.1. Image pre-processing

Because we are not experts in the field and there is also no clear consensus in the literature on whether specific regions of the breast need to be examined following a concrete criterium to diagnose breast tumors, we will segment the whole mammal gland as our SOI, and extract features from the whole breast to look for some patterns that help us classify mammographies between healthy and malignant tumors.

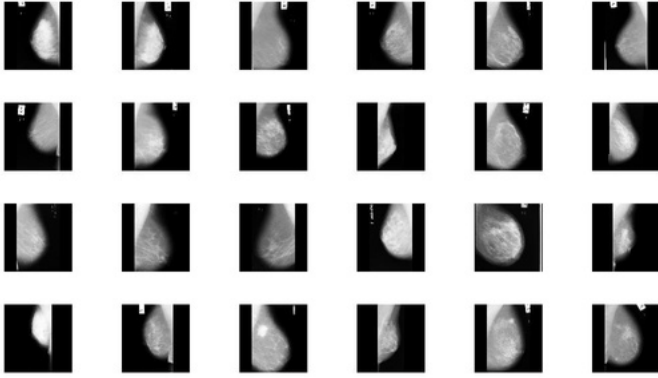


Figure 2. Original dataset of given images from [4].

Some of the original images we were given (Figure 2) contain labels utilized during the medical image modality apart from identification numbers for the patients (Figure 3). In order for these artifacts not to affect the segmentation process, a first pre-processing, including the cleaning of the images, is performed to remove them.

Segmentation of the whole breast will be conducted by looking at the intensity levels from the pixels in the images. Some of them included part of the chest. We do not want either to segment the chest but as it has similar intensity values as parts in the mammal gland, they will be also cleaned before the segmentation process from the images (Figure 3).

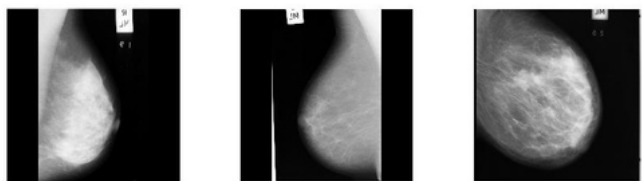


Figure 3. Images *mdb004.pgm*, *mdb009.pgm*, and *mdb148.pgm* from left to right. The left and middle ones show chest capture. All three collect labels and ID numbers. The middle and right ones collect artifacts in the borders of the image. All of them are to be cleaned up with pre-processing of the images.

This cleaning-up approach is firstly performed by using the function *roipoly()*, which allows us to generate a mask above

the image, being value 1 in the selected region of interest (ROI) and 0 elsewhere.

The image containing the generated mask is the same size as the one where it was applied. This allows us to perform element-wise multiplication. That way, just the pixels corresponding to 1 in the mask will be maintained in the original image, whereas the others will go to 0 intensity value.

The technique we follow is to select our ROI in *roipoly* as the region we want to clean up (the artifacts) instead of the SOI that will be actually segmented later. Therefore, before computing the product, we need to invert the generated mask so that we select in the original image everything but the ROI from the *roipoly*. An example of this cleaning-up process is shown in Figure 4 for image “*mdb009.pgm*”, where we have cleaned the chest, the labels, and the white borders.

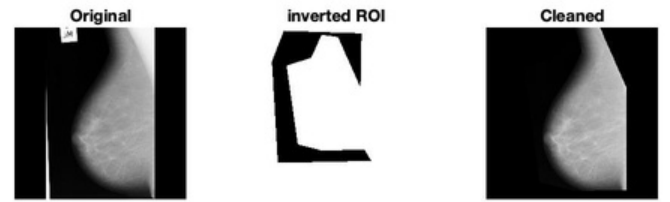


Figure 4. Cleaning-up process for image *mdb009.pgm*. Original image from [4], selected ROI to be cleaned in the image (already inverted to select everything but that ROI) (padded ROI), and cleaned deppadded resulting image, are shown from left to right. Labels, chest, and white borders are cleaned up on the right.

The original images were 0-padded before applying *roipoly()*. Otherwise, when trying to select the ROI, problems regarding the closing of the mask were raised. This padding was then subtracted to recover the original size of the image for the cleaned once element-wise multiplication had already been performed.

An adjustment for the intensity values of the cleaned image with *imadjust()* is carried out before saving it with *imwrite()* in *.jpg* format. That way, all images extend their pixel's intensity values from 0 to 1 so that they can be similarly visualized. The resulting pre-processed images are collected in Figure 5.

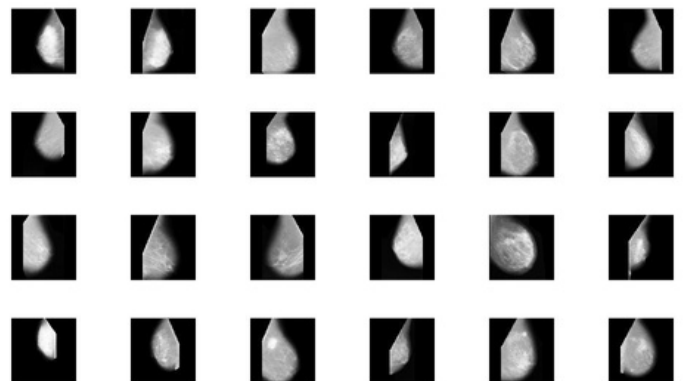


Figure 5. Collection of cleaned images achieved with the whole automatized process explained in this section. There are no labels, chest, or white borders.

It is important to highlight that this whole process of cleaning has been automatized within a homemade function in MATLAB gathered in *Appendix. Processing and Segmentation Code*.

These images are ready for our SOI (the whole breast) to be segmented. In the following section, we explain how we performed this.

2.2. Segmenting the structure of interest (SOI).

The library that is going to be used for feature extraction, needs the region in the image where we want to extract them from. As we stated before, we are interested in features from the whole breast. Therefore, its segmentation (our SOI) is needed.

We will proceed with this segmentation by selecting the areas in the cleaned images that differ from others because of differences in pixel intensity values. The whole code is also implemented in the function introduced before, which is in *Appendix. Pre-processing and Segmentation Code*.

Thanks to the fact that our original images have been cleaned, the SOI (the whole mammal gland) has different intensity values everywhere with respect to the rest of the image, which is in fact the background of the image. Therefore, only one threshold is needed to select the pixels that are above or below that specific intensity value to accomplish segmentation.

We could undergo the selection of this threshold for each of the images manually by looking at their histograms. However, because we want our model to be the most automatized possible, the function *graythresh()* is used instead. This function computes the threshold from a grayscale image, as ours, by using Otsu's method. This method chooses a threshold that minimizes the interclass variance of the white and black pixels passed through the threshold [10].

Once it is computed, we just apply to the cleaned image the condition of its pixels being 1 if their intensity values are above the given threshold, or 0 otherwise. We end up with binary images that are white (=1) inside the SOI and black (=0) outside; in the background. *Figure 6* presents some segmented breasts, SOIs, using the aforementioned approach.

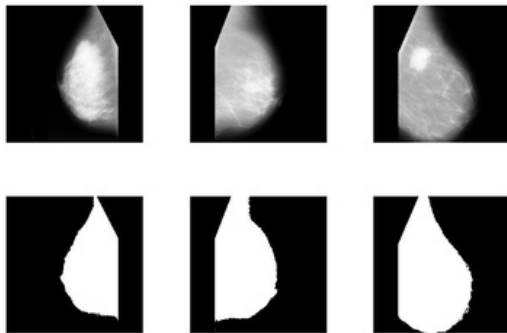


Figure 6. Cleaned *mdb005.pgm*, *mdb014.pgm*, and *mdb184.pgm* images in the superior row from left to right. The corresponding segmented in the row below.

Ideally, the segmented images should be smooth-edged. However, some of them (*Figure 7*) present irregularities. This is due to the fact that the original images are blurred in the edge regions, not leading to a sharp transition between the background and the breast but to a smooth transition. When binarizing the image, this results in a fits-and-spurts mask.

One way to avoid this problem would be by applying morphological operators (MO) such as erosions and dilations, or combinations such as closing and opening. We tried on some of the more irregular masks shown in *Figure 7*. Nonetheless, because our model is automatized, we would need to use a defined MO for refining all the masks. Each mask needs actually a specific MO to be selected by trial and error. In our case, we have 24 masks, and even though it would be very tedious work, it could be done manually. Nevertheless, as



Figure 7. Segmented from cleaned images corresponding to *mdb011.pgm*, *mdb026.pgm*, and *mdb148.pgm* images, from left to right. They show imperfections in the edges of the mask.

our aim is to develop a model that generalizes big image datasets, this could not be implemented without affecting the cost of the model.

In the end, these sharp peaks appear just at the edges of the mask, where in fact there is not much information to be extracted as it corresponds to the subdermal fat layer. For this reason and considering the aforementioned with respect to generalization, no MO will be applied to our masks. That way, the resulting masks including the segmentation of the structure of interest, which is the whole breast, are illustrated in *Figure 8*.

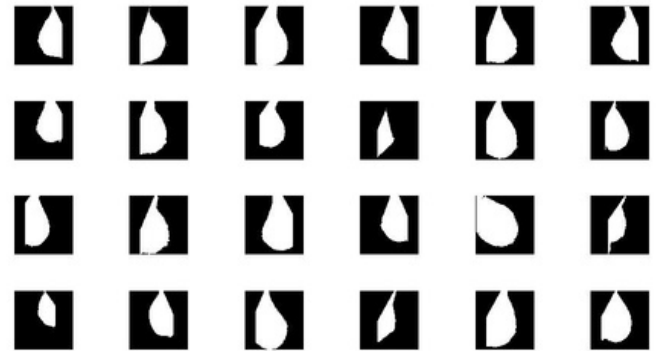


Figure 8. Collection of segmented images achieved with the whole automatized process explained in this section. The segmented structures are the whole breasts.

These masks, together with the cleaned images in *Figure 5*, are saved in .jpg format from MATLAB and ready to be used for the feature extraction explained in the next section.

2.3. Extraction of image features (radiomics)

For the purpose of obtaining the features from the data set, we will use *pyradiomics*. We think that features related to textures and intensity levels are of special importance. Thus, they are the ones to be extracted. *Pyradiomics* is a python open-source package made for image analysis and feature extraction [7], to be installed from the command line. Our supervisor Masias i Bruns Mireia helped us with its proper installation following the steps provided on the *pyradiomics* original website [7].

A conda environment was specifically created containing the mentioned package and a compatible version of python (3.8.6). This environment is then accessible from the anaconda browser and used to perform the feature extraction from the cleaned images based on the SOI marked by the segmented ones.

To understand the way of operating of *pyradiomics*, we first underwent some examples from the original website and got inspired by those so as to develop the spyder code that will extract our features of interest from the structures of interest. The final code is attached in *Appendix. Pyradiomics Code*.

For proper functioning of the code, we also installed, in the same environment, some libraries such as *os*, *six*, and *SimpleITK* as stated in the examples.

Some problems arose when we had to understand how to define the feature extractor using *params* dictionary. As its name indicates, it is a dictionary containing the parameters needed for the radiomics feature's extraction configuration from a medical image. These parameters report on how the data in the image must be processed. They include, among others, size of the masks used for feature extraction, filters to be applied before the extraction, how to calculate some of the features, etc.

Two possible solutions are available to solve this problem:

- Use *params* by default
- Specify a new *params*

For simplicity, we use *params* by default. It is defined in a python file found in one of the example folders from the pyradiomics original website [7]. It is written as '*Params.yaml*'.

Once this code is developed, it is ready to be used so as to obtain the features for each single image pair (the original one and the segmented one) of all the data in the folder.

The features from each image in the SOI, are saved in dictionaries that in turn are saved in a list named "results". In this way, we finally obtain a list of dictionaries where each one corresponds to the features extracted from each image in the corresponding SOI; the whole breast.

The whole process of feature extraction is implemented for all the images via a loop in the code. Nevertheless, analyzing the variables at the end of the process, we realized that there are a lot of them useless for our classification aim. They have no relevance and do not contain any useful information for our project. Those variables are mostly strings, dictionaries, or cryptographic codes as well as versions of python and pyradiomics, dimensional information, and others. In fact, we are mostly interested in textures or intensity levels in the SOI. They were extracted from the dictionaries using the *pop* function. A total of 91 features per image have been extracted from the available 115 features classified between first-order, second-order, Gray Level Co-Occurrence Matrix, Gray Level Size Zone Matrix, Neighboring Gray Tone Difference Matrix, and Gray Level Dependence Matrix.

Finally, because we already know whether the given mammographies correspond to cancer/no cancer, we implemented it as an attribute of our images. All the images with a name file of the form OXX are normal ones; classified as 0 class, and oncologic otherwise; classified as 1 class.

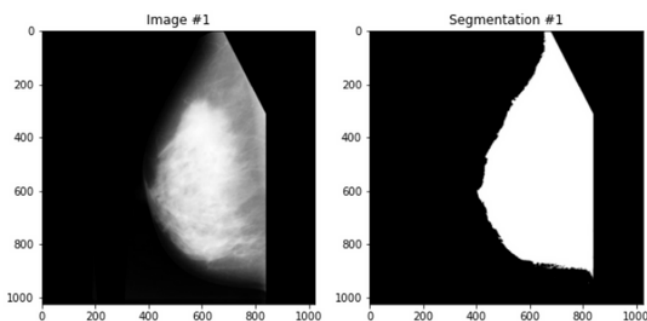


Figure 9. First pair of images plotted for visual control of the code in python.

To visualize what is the code running and if it takes the images correctly and correctly paired with its mask, plots as the one in *Figure 9* were obtained in *python 3.8.6* [6].

Pandas library is used also for purpose of generating a .csv file at the end of the code to save all the extracted features. This .csv file is automatically saved in the same folder as the script and written as 'data.csv' (*Figure 10*).

Finally from a .txt opening, we add the last modification in our feature extraction data introducing a label in the first column as 'name'.

[illegible]

Figure 10. generated data file .csv with the selected features.

2.4 Statistical comparison between groups

Once obtained the features, we will develop different techniques to visualize possible correlations within both classes and whether there is a way to distinguish malignant tumors from healthy mammal glands. We will approach both supervised and unsupervised learning and compare them to see whether one performs better.

However, we want first all the extracted features to be normalized to be on the same scale because it helps the algorithms that will be tested to be more optimal and efficient. This normalization is performed with the *Normalize* filter in *Weka 3.8.6* [8].

For the first approach, the supervised learning algorithms, we will make use of the fact that we actually know how the images are classified by experts. Therefore, some machine learning (ML) algorithms are going to be tested following supervised learning using the software *Weka 3.8.6* and 10-fold cross-validation. This model's goal is to correctly classify the instances given (each of the images) depending on the extracted features and look for some radiomics patterns and characteristics that allow this classification.

One of the simplest algorithms in supervised machine learning models is a Decision Tree. We will develop and train this model with our normalized dataset extracted from pyradiomics. Moreover, we will first perform a feature selection process with an Information Gain filter (*InfoGain* in *Weka*) and Subset Evaluation filter, to reduce the dimensionality of the dataset and try to avoid overfitting.

For the second approach, Principal Component Analysis (PCA) is to be performed. It consists of an unsupervised learning technique inside ML that aims to transform a set of correlated variables into a non-correlated and smaller set. These last variables are known as the Principal Components and they are selected depending on their variance. PCA works by identifying the directions in which the data varies the most (the highest variance); orthogonal to each other. Once the axes are found, the data is projected onto them, reducing the dimensionality to the desired number of features while preserving as much variance as possible [11].

Different from the supervised learning approach performed first, here no labels are considered to guide the learning of the model. Statistical techniques are used instead to find patterns and structures inside the data.

We will use *python 3.8.6* and the libraries *PCA* and *Scikit-Learn* to obtain an algorithm that reduces the dimensionality of our dataset to 2 features so that we can visualize the data with the maximized variance and infer if there are actually some statistical differences that allow us to differentiate the healthy mammographies from the cancerous ones.

3. Results and discussion

Once the features of interest have been extracted by using pyradioms, two different approaches have been conducted to compare them on their performance in looking for patterns or features that clearly help in the diagnosis of malignant tumors on the breasts. That way, automatization of the diagnosis could be implemented in the clinical world.

The performed feature selection carried out on *Weka* with *InfoGain* filter in order to diminish the dimensionality of the supervised learning model showed that three of the extracted features are somehow informative so as to classify the instances. The information they provide is relatively low though. These radiomics are *original_glcm_Correlation*, *original_glszm_LowGrayLevelZoneEmphasis*, and *original_glszm_SmallAreaLowGrayLevelEmphasis* (Figure 11). In fact, they are the same features selected as the ones that will help us classify our instances with the other proposed filter: Subset Evaluation (*SubsetEval* in *Weka*).

Ranked	attributes:
0.541	74 original_glszm_SmallAreaLowGrayLevelEmphasis
0.42	69 original_glszm_LowGrayLevelZoneEmphasis
0.344	30 original_glcm_Correlation
0	29 original_glcm_Contrast
0	91 original_gldm_SmallDependenceLowGrayLevelEmphasis
0	28 original_glcm_ClusterTendency
0	27 original_glcm_ClusterShade
0	22 original_firstorder_Uniformity
0	24 original_glcm_Autocorrelation
0	25 original_glcm_JointAverage
0	26 original_glcm_ClusterProminence

Figure 11. Screenshot of the results when applying *InfoGain* filter in *Weka*, for feature selection. Radiomics are sorted by the information gain they provide. Just the first three provide some. The rest (which are below the ones shown) give 0 info.

Selecting then these three features, different 10-fold cross-validation models are trained with our data following this supervised approach. The trained models are Decision Tree (J48 in *Weka*), and K-Nearest Neighbor (with $k=3$) (IBK in *Weka*). The corresponding confusion matrices are collected in Table 1 and Table 2, for each of the models.

Table 1. Confusion matrix for the model of decision trees with feature selection by *InfoGain* filter for cross-validation.

Healthy	Malignant	← classified as
15	1	Healthy
3	5	Malignant

Table 2. Confusion matrix for the model of K-NN ($k=3$) with feature selection by *InfoGain* filter for cross-validation.

Healthy	Malignant	← classified as
16	0	Healthy
3	5	Malignant

We can observe that the classifications are absolutely awful. They perform well in healthy cases but not in malignant ones,

which in fact are more important to be well-classified than the healthy ones. From a clinical point of view, it is better to have false positives than false negatives.

If we look at the specific generated tree in the Decision Tree model, the classification is being undergone just by looking at one of the features; the *original_glszm_Small Area Low Gray Level Emphasis* (Figure 12). This does not make sense at all.

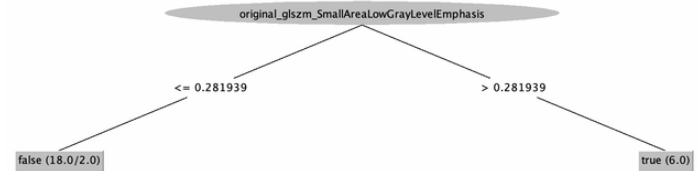


Figure 12. Screenshot of the resulting decision tree generated by the trained model.

It is relevant to notice the fact that we have extracted a high number of features (92 including the class) for a dataset of only 24 instances. Trying to find correlations with this proportion of features and examples will lead to overfitting and the misclassification of our instances. Moreover, using ML algorithms with such low data does not perform normally well, as in our case. We will need way more data to train a model that is later looked for generalization with a test set never seen before; something impossible now with just 24 instances.

The second approach, the one using unsupervised learning PCA, aims to reduce the dimensionality of our set instead of using the Decision Trees algorithm as we did previously. This method does so and allows us to visualize whether there are differences among the classes at the same time as we said before in 2.4. The results of the PCA are shown in Figure 13.

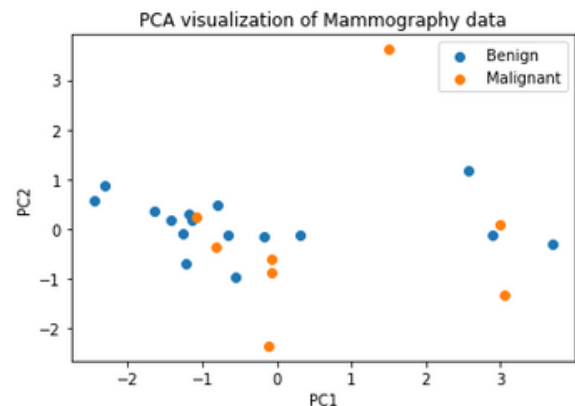


Figure 13. PCA 2D visualization of the features extracted from the mammographies.

As we can see, even after applying PCA and reducing the dimensionality to 2 features (Principal Component 1 and 2) with the highest variance possible, the two classes show no visible clustering, and therefore we cannot differentiate the samples with malignant tumors from the benign ones given the current instances and features.

Since the principal components are those that maximize the variance of the data, a high enough number of instances is essential to correctly perform the PCA and reduce the dimensionality, so an increase in the number of samples may lead to a conclusive result with visible statistical correlations.

4. Conclusions

In this work, we tried to face a clinical challenge as it is the automatization of breast cancer detection. Nonetheless, after the successful segmentation and radiomics extraction we started to struggle with the statistical classification.

As we commented in the previous section, our supervised machine-learning models have experienced overfitting with the inputs we gave. Our table of radiomics has 24 samples but 92 features. Therefore, even though we give the label of each instance, the algorithm can't find a manner to classify this into two groups. A possible solution for this would be to extend the dataset or reduce the features to more descriptive ones. However, the more suitable solution is a combination of both.

Afterward, we tried the reduction of dimensionality. Nevertheless, as we saw in Figure 13, there is no conclusive information. The dimensionality could not have been reduced with these features for a similar reason to the one we had in ML.

In conclusion, we could not perform the second part of the study as well as we expected, probably because of the size of the dataset. However, we could check that with just a few samples we can't carry out a thriving statistical study.

For future works, we recommend more automatization in the pre-processing of the images, because ours performs the cleaning up by selecting manually the ROI to be cleaned in each of them. If we used a bigger dataset we could not spend that much time working with the function *roipoly()*. Moreover, we could also try to extract different features to see if there is a possibility to reduce the group selection.

Finally, we conclude that with the tools we have right now, the performed algorithm is not competent enough for its implementation in clinical diagnosis. Nevertheless, this project offered us a good perspective and the basic knowledge of image processing and diagnostics for future projects and courses.

5. References

- 1- El cáncer de mama en España - GEICAM - Investigación en Cáncer de Mama [Internet]. GEICAM - Investigación en Cáncer de Mama. 2016 [cited on March the 10th 2023]. Disponible en: <https://www.geicam.org/sala-de-prensa/el-cancer-de-mama-en-espana>
- 2- Cáncer de mama metastásico - Estadísticas [Internet]. Cancer.net. 2017 [cited on March the 10th 2023]. Disponible en: <https://www.cancer.net/es/tipos-de-c%C3%A1ncer/c%C3%A1ncer-de-mama-metast%C3%A1sico/estad%C3%ADsticas>
- 3- Sistema. Cancer de mama - SEOM: Sociedad Española de Oncología Médica © 2019 [Internet]. Seom.org. 2023 [cited on March the 10th 2023]. Disponible en: <https://seom.org/info-sobre-el-cancer/cancer-de-mama?start=1>
- 4- J Suckling et al (1994) "The Mammographic Image Analysis Society Digital Mammogram Database" Excerpta Medica. International Congress Series. 1069 pp375-378.

5- MATLAB version: 9.11.0 (R2021b), Natick, Massachusetts: The MathWorks Inc.; 2022.

6- Van Rossum Guido. Python 3.8.7. Netherlands; 2022.

7- Welcome to pyradiomics documentation — pyradiomics v3.0.1.post15+g2791e23 documentation [Internet]. Readthedocs.io. [cited on March the 10th 2023]. Disponible en: <https://pyradiomics.readthedocs.io/en/latest/>

8- Weka 3.8.6: Data Mining Software in Java. (2016). Retrieved from <https://www.cs.waikato.ac.nz/ml/weka/>

9- Principal components analysis explained for dummies [Internet]. Programmathically - A Blog on Building Machine Learning Solutions. Programmathically; 2022 [cited on March the 10th 2023]. Disponible en: <https://programmathically.com/principal-components-analysis-explained-for-dummies/>

10- Umbral de imagen global usando el método de Otsu - MATLAB graythresh - MathWorks España. (s. f.). <https://es.mathworks.com/help/images/ref/graythresh.html>

11- Rodrigo, J. A. (s. f.). Análisis de Componentes Principales (Principal Component Analysis, PCA) y t-SNE. https://www.cienciadedatos.net/documentos/35_principal_component_analysis

Appendix

Preprocessing and segmentation code (matlab)

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% SEGMENTATION AND CLEANING
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

names = {'003','004','006','007','008','009','011','014','016','018','020','022','024','026','027','029','148','178','179','181','184','186','202','206'}
for i = names
    name = 'mdb'+string(i)
    segmenting(name)
end

function [] = segmenting(name)

% Loading the image
im1 = im2double(imread(string(name)+'.pgm'));

% Padding the image
[M,N] = size(im1);
im1_pad = zeros(1500);im1_pad(100:(99+M),100:(99+N))=im1;

% Creating the mask
mask = roipoly(im1_pad);
mask1 = mask<1;
imwrite(mask1,'/Users/alvaroborachok/Documents/MATLAB/AIB/proyecto/masks/'+string(name)+'_mask.jpg');

% Visualizing the roipoly mask
figure;imshow(mask1);

% Cleaning
im1=(im1_pad.*mask1);

% Deppading
im_clean = im1(100:(99+M),100:(99+N));

% Showing cleaned and adjusting
im_clean = imadjust(im_clean,[min(min(im_clean));max(max(im_clean))],[0;1]);
figure;imshow(im_clean);
imwrite(im_clean,'/Users/alvaroborachok/Documents/MATLAB/AIB/proyecto/a_pyradiomics en jpg/'+string(name)+'_clean.jpg');

% Thresholding
m = graythresh(im_clean);
imwrite(im_clean>m,'/Users/alvaroborachok/Documents/MATLAB/AIB/proyecto/a_pyradiomics en jpg/'+string(name)+'_thr.jpg');

end
```

Pyradiomics code (python)

```
import os
import six
import SimpleITK as sitk
from radiomics import featureextractor
import matplotlib.pyplot as plt
import pandas as pd

filelist = ['003','004','006','007','008','009','011','014','016','018','020','022','024','026','027','029','148','178','179','181','184','186','202','206']
typelist = ['D','D','F','G','G','F','F','G','G','G','G','G','G','F','F','G','F','G','D','G','F','G','D','F']
results=[]
for i in range(0,len(filelist)):
    name1= 'mdb'+filelist[i] + '_clean.jpg'
    name2= 'mdb'+filelist[i] + '_thr.jpg'

    image_1 = sitk.ReadImage(name1)
    label_1 = sitk.ReadImage(name2)

    params = os.path.join(os.getcwd(), 'Params.yaml')

    extractor = featureextractor.RadiomicsFeatureExtractor(params)
    result_1 = extractor.execute(image_1, label_1)
    result_1.pop('diagnostics_Configuration_EnabledImageTypes')
    result_1.pop('diagnostics_Configuration_Settings')
    result_1.pop('diagnostics_Image-original_Size')
    result_1.pop('diagnostics_Image-original_Spacing')
    result_1.pop('diagnostics_Mask-original_BoundingBox')
    result_1.pop('diagnostics_Mask-original_CenterOfMass')
    result_1.pop('diagnostics_Mask-original_CenterOfMassIndex')
    result_1.pop('diagnostics_Mask-original_Size')
    result_1.pop('diagnostics_Mask-original_Spacing')
    result_1.pop('diagnostics_Image-original_Hash')
    result_1.pop('diagnostics_Image-original_Dimensionality')
    result_1.pop('diagnostics_Mask-original_Hash')
    result_1.pop('diagnostics_Versions_Numpy')
    result_1.pop('diagnostics_Versions_PyRadiomics')
    result_1.pop('diagnostics_Versions_Python')
    result_1.pop('diagnostics_Versions_PyWavelet')
    result_1.pop('diagnostics_Versions_SimpleITK')

    result_1["type"]=typelist[i]
    if filelist[i][0]=='0':
        result_1["Class"]='0'
    else:
        result_1["Class"]='1'
    results.append(result_1)

    plt.figure(figsize=(10,10))
    # First image
    plt.subplot(1,2,1)
    plt.imshow(sitk.GetArrayFromImage(image_1), cmap="gray")
    plt.title("Image #1")
    plt.subplot(1,2,2)
    plt.imshow(sitk.GetArrayFromImage(label_1),cmap="gray")
    plt.title("Segmentation #1")

    plt.show()

df = pd.DataFrame(results,filelist)
df.to_csv("data.csv")
```

Code for generating the first 8 figures (MATLAB)

```
%%%%%%%%%%%%%
% figure 2
%%%%%%%%%%%%%
names = {'003','004','006','007','008','009','011','014','016','018','020','022','024','026','027','029','148','178','179','181','184','186','202','206'};
i = 1;
figure(1);
for name = names
    im1 = im2double(imread('mdb'+string(name)+'.pgm'));
    subplot(4,6,i);imshow(im1,[]);
    i = i+1;
end

%%%%%%%%%%%%%
% figure 3
%%%%%%%%%%%%%
names = {'004','009','148'};
i = 1;
figure(2);
for name = names
    im1 = im2double(imread('mdb'+string(name)+'.pgm'));
    subplot(1,3,i);imshow(im1,[]);
    i = i+1;
end

%%%%%%%%%%%%%
% figure 4
%%%%%%%%%%%%%
name = '009';
original = im2double(imread('mdb'+string(name)+'.pgm'));
roipo = im2double(imread('mdb'+string(name)+'_mask.jpg'));
cleaned = im2double(imread('mdb'+string(name)+'_clean.jpg'));
figure(3);
subplot(131);imshow(original,[]);title('Original')
subplot(132);imshow(roipo,[]);title('inverted ROI')
subplot(133);imshow(cleaned,[]);title('Cleaned')

%%%%%%%%%%%%%
% figure 5
%%%%%%%%%%%%%
names = {'003','004','006','007','008','009','011','014','016','018','020','022','024','026','027','029','148','178','179','181','184','186','202','206'};
i = 1;
figure(4);
for name = names
    subplot(4,6,i);imshow(im2double(imread('mdb'+string(name)+'_clean.jpg')));
    i = i+1;
end

%%%%%%%%%%%%%
% figure 6
%%%%%%%%%%%%%
names = {'003','014','184'};
i = 1;
figure(5);
for name = names
    subplot(2,3,i);imshow(im2double(imread('mdb'+string(name)+'_clean.jpg')),[]);
    subplot(2,3,i+3);imshow(im2double(imread('mdb'+string(name)+'_thr.jpg')),[]);
    i = i+1;
end

%%%%%%%%%%%%%
% figure 7
%%%%%%%%%%%%%
names = {'011','026','148'};
i = 1;
figure(6);
for name = names
    subplot(1,3,i);imshow(im2double(imread('mdb'+string(name)+'_thr.jpg')));
    i = i+1;
end

%%%%%%%%%%%%%
% figure 8
%%%%%%%%%%%%%
names = {'003','004','006','007','008','009','011','014','016','018','020','022','024','026','027','029','148','178','179','181','184','186','202','206'};
i = 1;
figure(7);
for name = names
    subplot(4,6,i);imshow(im2double(imread('mdb'+string(name)+'_thr.jpg')));
    i = i+1;
end
```

PCA code (python)

```
import pandas as pd
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

# Load the dataset
data2 = pd.read_csv('finaldata.csv')

y = data2["Class"];
X = data2.loc[:, data2.columns!='Class']

pca = PCA(n_components=2) #Create a PCA object (retain 2 principal components)
pca.fit(X) #Fit the PCA object into the data
X_pca = pca.transform(X) #Perform the PCA

#Plot the result
plt.figure()
plt.scatter(X_pca[y==0, 0], X_pca[y==0, 1], label='Benign')
plt.scatter(X_pca[y==1, 0], X_pca[y==1, 1], label='Malignant')
plt.title("PCA visualization of Mammography data")
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.legend()
```

Team work description and distribution

Álvaro:

My contribution to the project has been the implementation of the code of the pre-processing of the images with Zaid's help and Paul's proposals as well as the training of supervised learning algorithms with Weka. In the paper, I was encharged of sections 2.1 and 2.2 as well as results and discussion of the ML-supervised algorithms.

Zaid:

My contribution to the project was the study of the pyradiomics library with Paul and Pau for its implementation. I also helped Álvaro a little bit with the pre-processing part. Finally, I read websites of breast cancer literature and wrote the introduction part.

Paul:

My role in this project was principally the implementation of pyradiomics together with Pau and Zaid for the .csv features extracted file. I also came up with ideas for the pre-processing implementations together with Álvaro. For the paper, I have been in charge of writing the pyradiomics part and the abstract.

Pau:

My contribution to the project has been to implement the pyradiomics code for feature extraction together with Paul and Zaid. Also, I have studied how PCA works and implemented it by writing a python code to reduce the dimensionality of the data.